Chapter 1

Queues

Imagine you are at a concert. You are arriving at around 1pm, but the line has been growing since about 3am that same day (wow!). Naturally, by way of timing, the people who arrived earliest are going to be able to enter the venue first, while the people you arrive closest to the show time will enter the venue last.

Assuming supplements like VIP packages or fast-track lines are not of concern here, we can summarize this type of line in a similar phrase: **First In, First Out**. The first people to arrive in the line are the first people out of the line, and, subsequently, into the venue. We call this way of organizing data *FIFO*.

Similar to stacks, we only need a handful of operations to build a queue.

[placeholder: people in line image (maybe)]

Every stack, no matter how it is built underneath, supports the same handful of operations:

- `enqueue(value)` put a new value at the front of the queue. You may see this as `enq`
- `dequeue()` remove and return the value at the front of the queue. You may see this as `deq`
- `peek()` look at the value on front of the queue
- `isEmpty()` true if the queue has nothing in it.

Similarly, again, to stacks, we can implement this two ways: a fixed-array based method and a linked-list method. Both have benefits and tradeoffs.

1.1 Array-Based Queue

We can build a queue with an array, there is the same major constraint as with stacks: we need to know the size of the queue in advance.

```
1  const int CAPACITY = 5;
2
3  struct ArrayQueue {
4      int data[CAPACITY];
5      int front;   // index of the front element; -1 means empty
```

```
    int count = 0; // for ensuring we don't exceed CAPACITY

};

bool enqueue(Queue& q, int value) {
    if (isFull(q)) return false;
    int back = (q.front + q.count) % MAX;  // wraps around
    q.data[back] = value;
    q.count++;
    return true;
}

bool dequeue(Queue& q, int& out) {
    if (isEmpty(q)) return false;
    out = q.data[q.front];
    q.front = (q.front + 1) % MAX;         // wraps around
    q.count--;
    return true;
}

bool peek(const Queue& q, int& out) {
    if (isEmpty(q)) return false;
    out = q.data[q.front];
    return true;
}
```

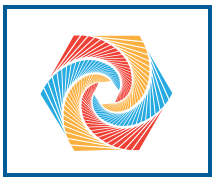
TODO: Explain why we need to wrap around the array (circular array) and how it is done. Simplify above code.

1.2 Linked-List Queue

This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.

Appendix A

Answers to Exercises



INDEX

FIFO, 1